

目录

×

01、Agent 的联网搜索效果

02、Agent 联网搜索的整体架...

03、搜索引擎 Provider 抽象

智谱搜索

SerpAPI

SearXNG

工厂自动选择

04、WebFetch 抓取器

HTTP 抓取

网络安全策略

HTML 正文提取

05、工具注册和 Agent 集成

06、配置和使用

最简配置（推荐）

切换搜索引擎

实际使用

07、如何把 PaiCLI 写到简历...

ending

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- [04、WebFetch 抓取器](#)
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- [07、如何把 PaiCLI 写到简历...
ending](#)

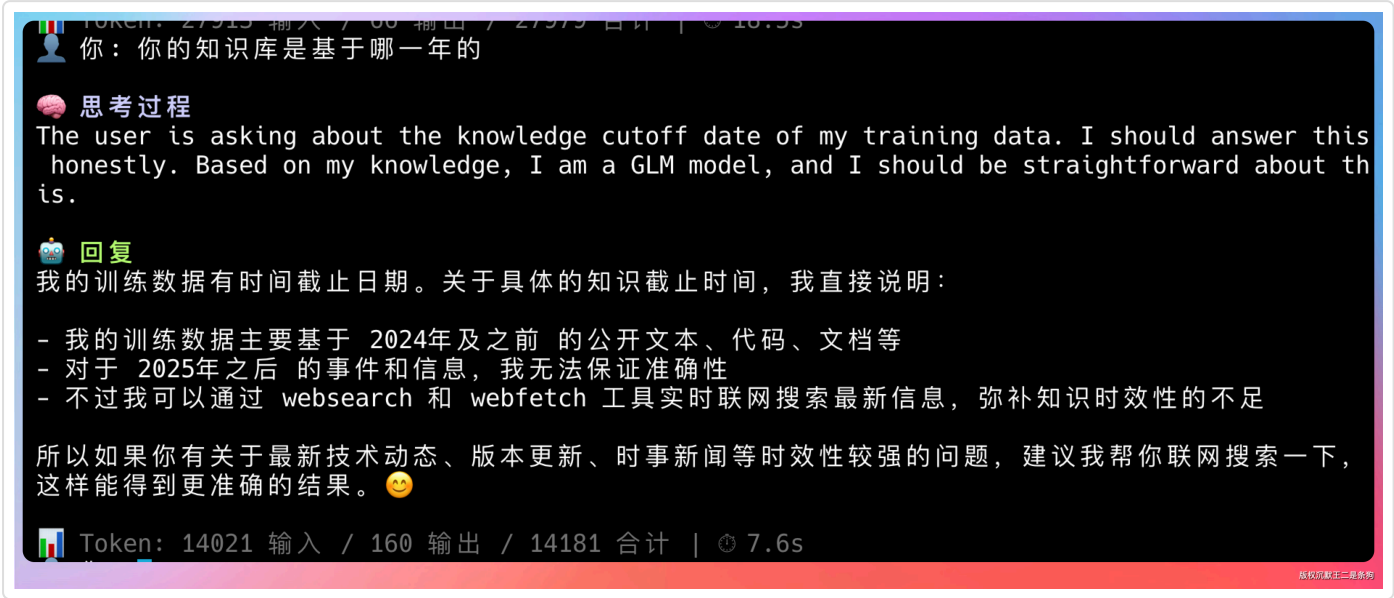
原创

手把手教你给 Agent 加上联网能力，WebSearch + WebFetch 让 Agent 知道世界发生了什么。

大家好，我是二哥呀。

大模型本身是没有联网能力的，他的知识库都是基于某一个时刻训练完成的。

像 GLM-5.1，通过 PaiCLI 去问的话，答案是基于 2024 年及之前的公开文本、代码、文档。



而现在已经是 2026 年 4 月 27 日了，这就意味着如果 Agent 没有联网能力的话，他对新事物是没有感知的。

那怎么给 Agent 加上联网能力呢？

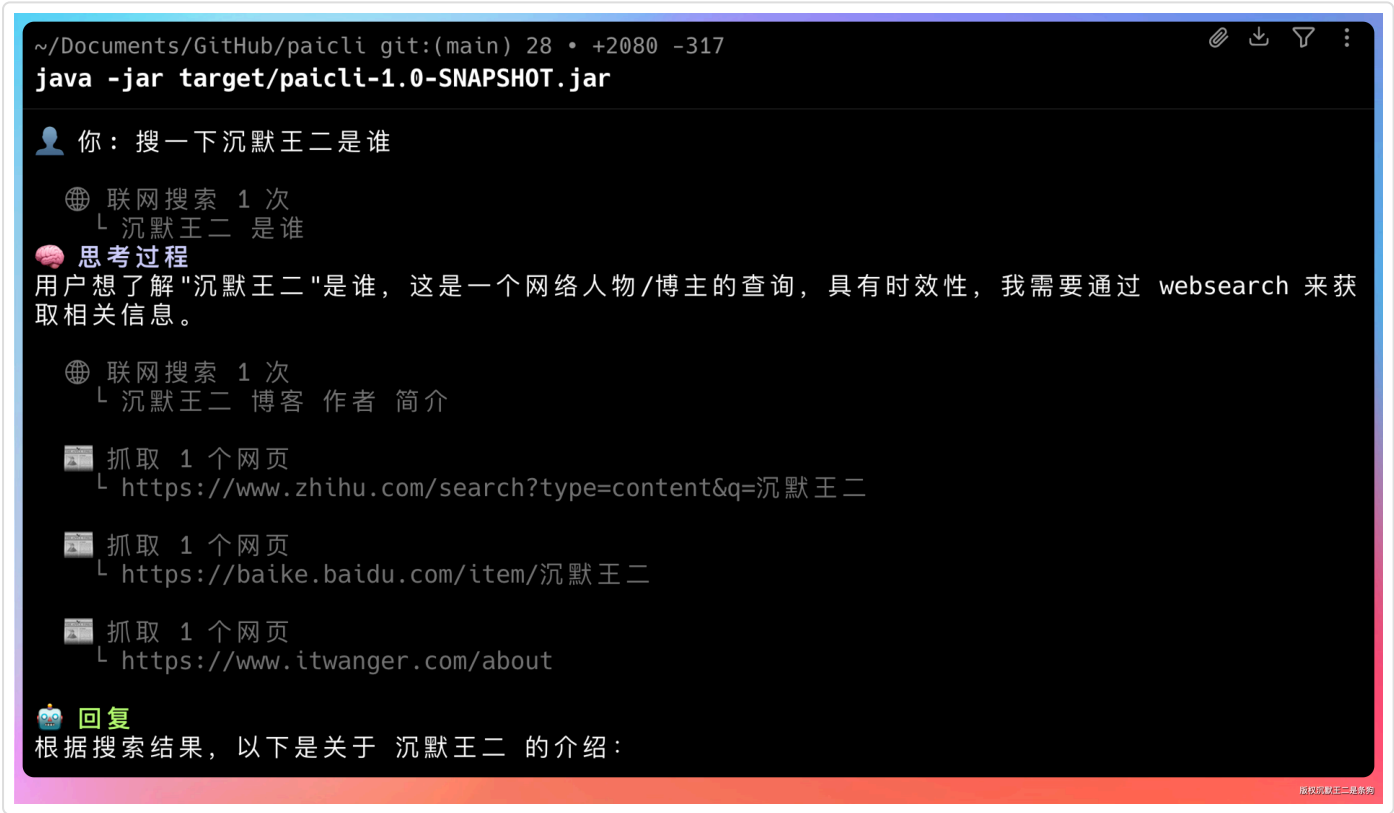
今天这篇内容我们就来完成他。

01、Agent 的联网搜索效果

先来看 PaiCLI Agent 加上联网能力之后能干什么。

在终端里输入提示词：“搜一下沉默王二是谁”。

PaiCLI Agent 会自动调用 `web_search` 工具，通过智谱搜索 API 获取相关信息。



然后把搜索结果整理成一段非常准确的回复：

沉默王二 是一位知名的技术博主、程序员，真实身份与笔名一致，活跃于技术社区和自媒体领域。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - 智谱搜索
 - SerpAPI
 - SearXNG
 - 工厂自动选择
- 04、WebFetch 抓取器
 - HTTP 抓取
 - 网络安全策略
 - HTML 正文提取
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - 最简配置（推荐）
 - 切换搜索引擎
 - 实际使用
- 07、如何把 PaiCLI 写到简历...ending

 回复

根据搜索结果，以下是关于 沉默王二 的介绍：

 沉默王二简介

沉默王二 是一位知名的中国技术博主、程序员，真实身份与笔名一致，活跃于技术社区和自媒体领域。

 主要经历

时间 / 事件

- 2008年

在大学期间学习 Java 编程

- 2010-2013年

南下苏州打拼，从普通程序员成长为 Team Leader

- 2014年

回到洛阳，一个人扛起公司所有技术，开启创业之路

- 2015年

开始在 CSDN 写博客，年末成为 CSDN 博客专家

- 2019年 4月

出版书籍 《Web全栈开发进阶之路》

- 2019年 11月

微信公众号「沉默王二」突破 1万 读者

 个人品牌

- 个人博客：www.itwanger.com

- 微信公众号：沉默王二（ID: cmower），定位"分享技术、感悟人生"

- CSDN 博客：CSDN 博客专家，长期输出高质量技术文章

版权沉默王二是真的

就连我的座右铭都给出来了：**没有什么可以使我停留，除了目的，纵然岸边有绿荫，有宁静的港湾，我是不系之舟。**这回答我给满分。

 个人品牌

- 个人博客：www.itwanger.com

- 微信公众号：沉默王二（ID: cmower），定位"分享技术、感悟人生"

- CSDN 博客：CSDN 博客专家，长期输出高质量技术文章

- 技术方向：主要专注于 Java 相关技术栈

 个人格言

| "没有什么可以使我停留，除了目的，纵然岸边有绿荫，有宁静的港湾，我是不系之舟。"

 特色

他自称："一个文章真特么有趣的程序员！"，以幽默风趣的写作风格在技术圈广受好评，文章风格轻松易读，深受初学者和进阶开发者喜爱。

版权沉默王二是真的

再来一个。输入提示词：“paicoding.com 今天有没有更新啊”。

这次 PaiCLI Agent 调用的是 `web_fetch` 工具，直接去抓取网页内容。

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)

[智谱搜索](#)

[SerpAPI](#)

[SearXNG](#)

[工厂自动选择](#)
- [04、WebFetch 抓取器](#)

[HTTP 抓取](#)

[网络安全策略](#)

[HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)

[最简配置（推荐）](#)

[切换搜索引擎](#)

[实际使用](#)
- [07、如何把 PaiCLI 写到简历...ending](#)


```
Token: 9697 输入 / 555 输出 / 10252 合计 | @ 50.8S
你: paicoding.com 今天有没有更新啊

📄 抓取 1 个网页
  ↳ https://paicoding.com

🧠 思考过程
The user is asking if paicoding.com has been updated today. Let me fetch the website to check.

📄 抓取 1 个网页
  ↳ https://paicoding.com/articles
The website loaded successfully. I can see recent articles but I need to check if there are specific dates. Let me look at the content more carefully. The content shows various articles but doesn't include specific dates. Let me try to fetch more details, maybe a specific page with dates, or check for today's updates.

📄 抓取 1 个网页
  ↳ https://paicoding.com/column/all

📄 抓取 1 个网页
  ↳ https://paicoding.com/article/list

📄 抓取 1 个网页
  ↳ https://paicoding.com/category

📄 抓取 1 个网页
  ↳ https://paicoding.com/sitemap.xml

Let me look at the sitemap data more carefully. I can see dates like 2026-04-27, 2026-04-26, 2026-04-25, 2026-04-23, etc. Today appears to be April 27, 2026 based on the timestamps.

Let me check the most recent articles. I can see:
```

它把教程、文章、sitemap 都抓了一个遍。

```
- https://paicoding.com/article/detail/2611530100684803 → 2026-04-26

Let me fetch that article to see what it is.

Actually, looking more carefully at the data:

For April 27: Only the homepage and column page were updated (sitemap was regenerated).
For April 26: article 2611530100684803
For April 25: articles 2611500627310594, 2611500627310593

📄 抓取 1 个网页
  ↳ https://paicoding.com/article/detail/2611530100684803
So the sitemap was regenerated today (April 27) but the latest article appears to be from April 26. Let me check that article.

Now I have a clear picture. Let me summarize the findings.

Today is April 27, 2026. Based on the sitemap:
- The homepage was updated on 2026-04-27 (sitemap regeneration)
- The latest actual article was published on 2026-04-26: "DeepSeek V4 来了，这波我真的上头（附技术拆解报告）"

So the sitemap was regenerated today but no new article was published today. The most recent article was from yesterday (April 26).

🗨️ 回复
根据我对 paicoding.com（技术派）的检查，以下是今天（2026年4月27日）的更新情况：

---

📄 paicoding.com 今日更新情况
-----
```

最终抓取到最新的文章是：《DeepSeek V4 来了，这波我真的上头（附技术拆解报告）》，昨天发布的。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - 智谱搜索
 - SerpAPI
 - SearXNG
 - 工厂自动选择
- 04、WebFetch 抓取器
 - HTTP 抓取
 - 网络安全策略
 - HTML 正文提取
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - 最简配置（推荐）
 - 切换搜索引擎
 - 实际使用
- 07、如何把 PaiCLI 写到简历...ending

● 今天（4月27日）没有发布新文章

不过网站的 sitemap 在今天有更新（首页和栏目页的索引刷新）。

● 最近一篇文章发表于昨天（4月26日）

最新文章标题：《DeepSeek V4 来了，这波我真的上头（附技术拆解报告）》

信息 / 详情

- 作者
管理员（二哥）

- 发布时间
2026年 04月 26日 06:33

- 阅读量
166

- 主要内容
将 DeepSeek V4 接入 PaiCLI，支持 GLM-5.1 和 DeepSeek V4 运行时一键切换

📅 近几天的更新记录

日期	文章数	内容方向
4月26日	1篇	DeepSeek V4 接入 + 技术拆解报告
4月25日	2篇	相关主题文章
4月23日	2篇 + 1个专栏更新	PaiCLI 系列
4月20日	专栏更新	-
4月16日	2篇	-
4月15日	2篇	-
4月13日	2篇 + 1个专栏	-

版权原主二哥原创

然后对网站整体做了一个非常细致的总结，包括最近的一些更新内容：

今天（4 月 27 日）暂无新文章发布，最近的一篇是昨天（4 月 26 日）关于 DeepSeek V4 的深度技术拆解文章。该站近期更新频率很高，几乎每隔 1-2 天就有新内容，主要集中在 PaiCLI、AI Agent、RAG、DeepSeek 等前沿 AI 技术方向。

💡 总结

今天（4月27日）暂无新文章发布，最近的一篇是昨天（4月26日）关于 DeepSeek V4 的深度技术拆解文章。该站近期更新频率很高，几乎每隔 1-2 天就有新内容，主要集中在 PaiCLI、AI Agent、RAG、DeepSeek 等前沿 AI 技术方向。

🇺🇸 Token: 44986 输入 / 1339 输出 / 46325 合计 | ⌚ 89.8s

版权原主二哥原创

这样 PaiCLI Agent 就具备了最基础的 WebSearch 和 WebFetch 能力，可以随时获取最新的网络信息。

不用手动打开浏览器搜索，不用复制粘帖网页内容，Agent 自己就能搞定。

接下来，我给大家详细讲讲是怎么实现的。

02、Agent 联网搜索的整体架构

PaiCLI 的联网模块放在 `com.paicli.web` 包下，一共 8 个类，分工明确。

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)

[智谱搜索](#)

[SerpAPI](#)

[SearXNG](#)

[工厂自动选择](#)
- [04、WebFetch 抓取器](#)

[HTTP 抓取](#)

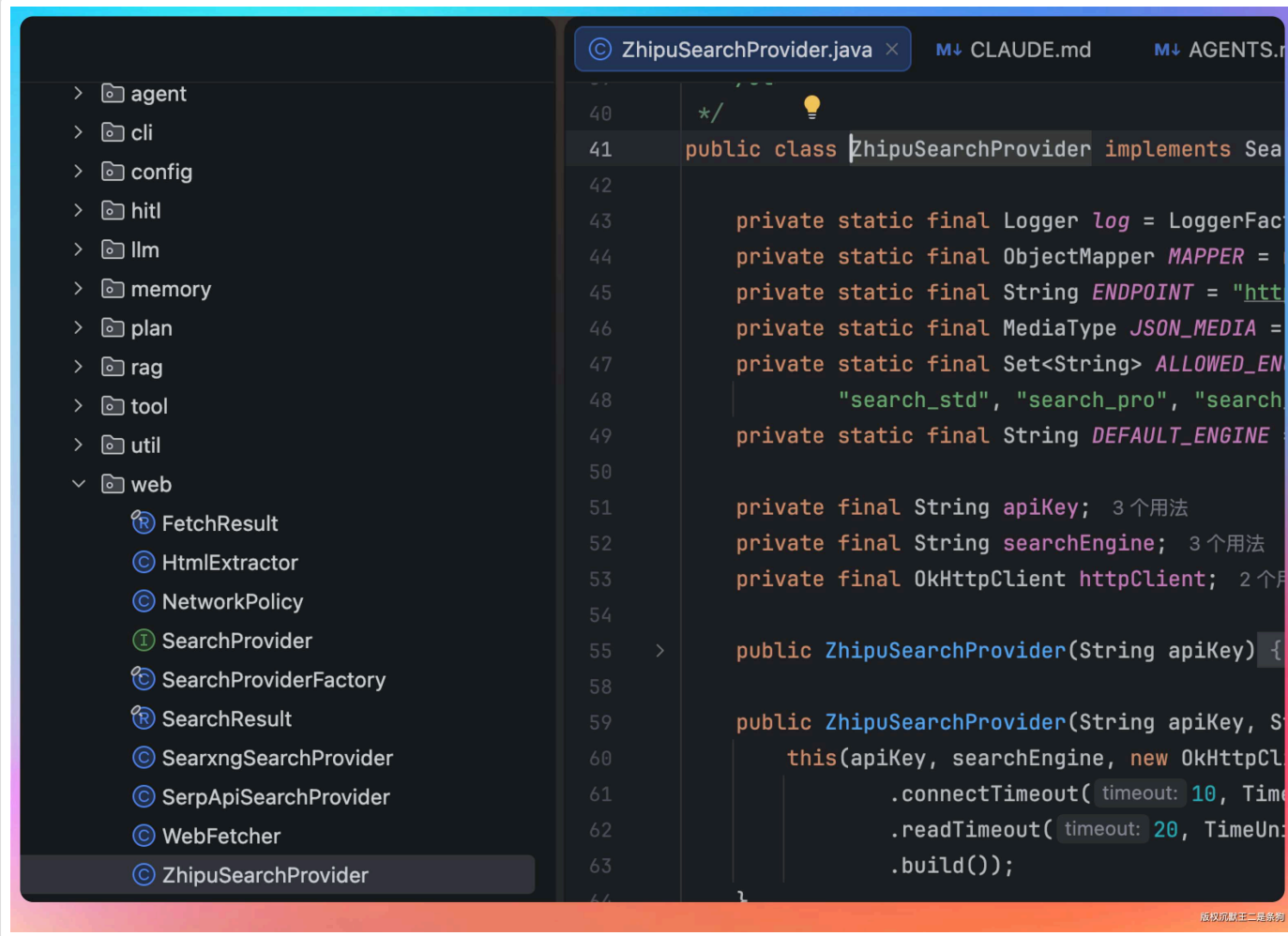
[网络安全策略](#)

[HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)

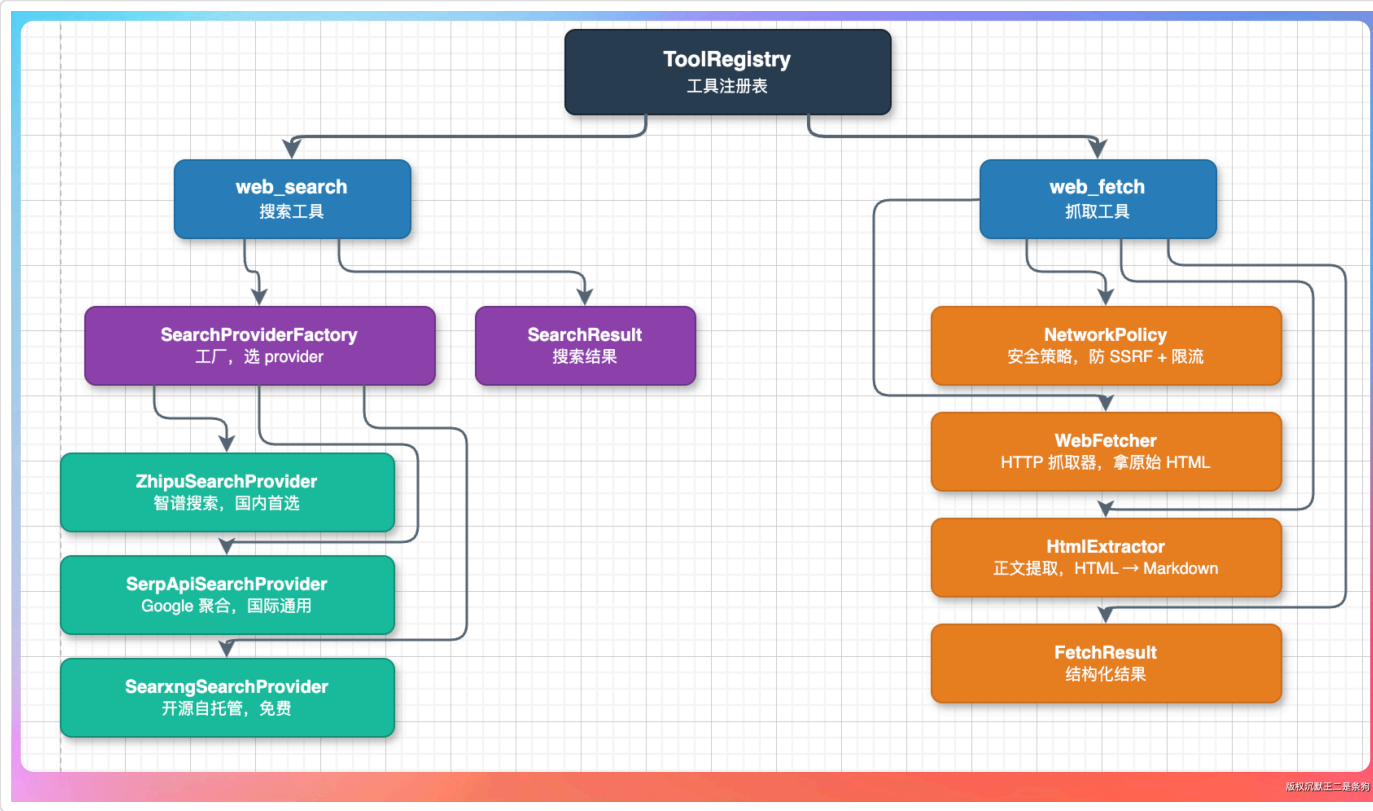
[最简配置（推荐）](#)

[切换搜索引擎](#)

[实际使用](#)
- [07、如何把 PaiCLI 写到简历...ending](#)



我画了一张依赖关系图，大家感受一下这个架构：



这里有两个设计决策值得说一下。

第一个，搜索和抓取是两个独立的工具，不是一个“联网工具”。

为什么？

因为它们的使用场景完全不同。

搜索是“我不知道去哪找”，抓取是“我知道 URL，帮我把内容拿回来”。

分成两个工具，Agent 可以根据用户意图自己选择用哪个，也可以先搜再抓（搜到 URL 后再 fetch 详情）。

第二个，搜索引擎做了 Provider 抽象。

三个实现类各有特点：智谱的联网搜索可以和 LLM 共用 API Key 零额外配置，SerpAPI 付费但开箱即用，SearXNG 开源免费但需要自己部署。

通过工厂模式自动选择。

为什么不直接在代码里写死用智谱？

目录

[01、Agent 的联网搜索效果](#)

[02、Agent 联网搜索的整体架...](#)

[03、搜索引擎 Provider 抽象](#)

[智谱搜索](#)

[SerpAPI](#)

[SearXNG](#)

[工厂自动选择](#)

[04、WebFetch 抓取器](#)

[HTTP 抓取](#)

[网络安全策略](#)

[HTML 正文提取](#)

[05、工具注册和 Agent 集成](#)

[06、配置和使用](#)

[最简配置（推荐）](#)

[切换搜索引擎](#)

[实际使用](#)

[07、如何把 PaiCLI 写到简历...](#)

[ending](#)

工具名称	简介	单价
Search-Std	智谱自研搜索基础版：速度快，高性价比	0.01元 / 次
Search-Pro	智谱自研搜索Pro版：搜索结果召回率更高	0.03元 / 次
Search-Pro-Sogou	搜狗搜索：耗时短，支持搜狗百科、搜狗问问检索	0.05元 / 次
Search-Pro-Quark	夸克搜索：搜索时效性强、覆盖多行业领域	0.05元 / 次

因为 PaiCLI 虽然主要面向国内用户，但不排除二哥装逼给海外用户用。

最重要的是，能教大家学到东西，比如策略模式 😊

做了 Provider 抽象之后，以后哪怕再加一个 Bing Search 或者 Tavily，也只需要加一个实现类和工厂里加一行判断就行了，不用动任何现有逻辑。

这就是策略模式的好处，开闭原则玩得很到位。

03、搜索引擎 Provider 抽象

先看接口定义，`SearchProvider` 只有四个方法：

java 复制代码

```
public interface SearchProvider {
    String name();           // provider 名称
    boolean isReady();       // 是否可用
    String unavailableHint(); // 不可用时的提示
    List<SearchResult> search(String query, int topK) throws IOException;
}
```

为什么要有 `isReady()` 和 `unavailableHint()`？

因为用户可能还没配置 API Key 就开始用了。

这时候不是抛异常让程序崩掉，而是友好地提示“请配置 XXX”。

这种防御式设计在 CLI 工具里特别重要，用户的环境千其百怪，你不能假设所有配置都是完美的。

智谱搜索

智谱搜索是我给 PaiCLI 选的默认 Provider，原因很简单：PaiCLI 主要面向国内 GLM 用户，智谱的搜索 API 和 LLM 推理共用同一个 `GLM_API_KEY`，不需要额外注册、额外付费、额外配置。

调用方式是 POST 请求到 `https://open.bigmodel.cn/api/paas/v4/tools/web_search`，请求体长这样：

java 复制代码

```
ObjectNode payload = MAPPER.createObjectNode();
payload.put("search_engine", searchEngine); // search_std / search_pro
payload.put("search_query", query);
payload.put("count", count);
payload.put("content_size", "medium");
```

智谱提供了四种搜索引擎可选：

- `search_std` 标准版 0.01 元/次，
- `search_pro` 增强版 0.03 元/次，
- 还有搜狗和夸克的 Pro 版各 0.05 元/次。

默认用 `search_std` 就够了，一次搜索一分钱，比 SerpAPI 便宜 5 到 10 倍。

目录

[01、Agent 的联网搜索效果](#)

[02、Agent 联网搜索的整体架...](#)

[03、搜索引擎 Provider 抽象](#)

[智谱搜索](#)

[SerpAPI](#)

[SearXNG](#)

[工厂自动选择](#)

[04、WebFetch 抓取器](#)

[HTTP 抓取](#)

[网络安全策略](#)

[HTML 正文提取](#)

[05、工具注册和 Agent 集成](#)

[06、配置和使用](#)

[最简配置（推荐）](#)

[切换搜索引擎](#)

[实际使用](#)

[07、如何把 PaiCLI 写到简历...ending](#)

整个抓取流程分三步：HTTP 请求拿原始 HTML → 安全检查 → 正文提取转 Markdown。

HTTP 抓取

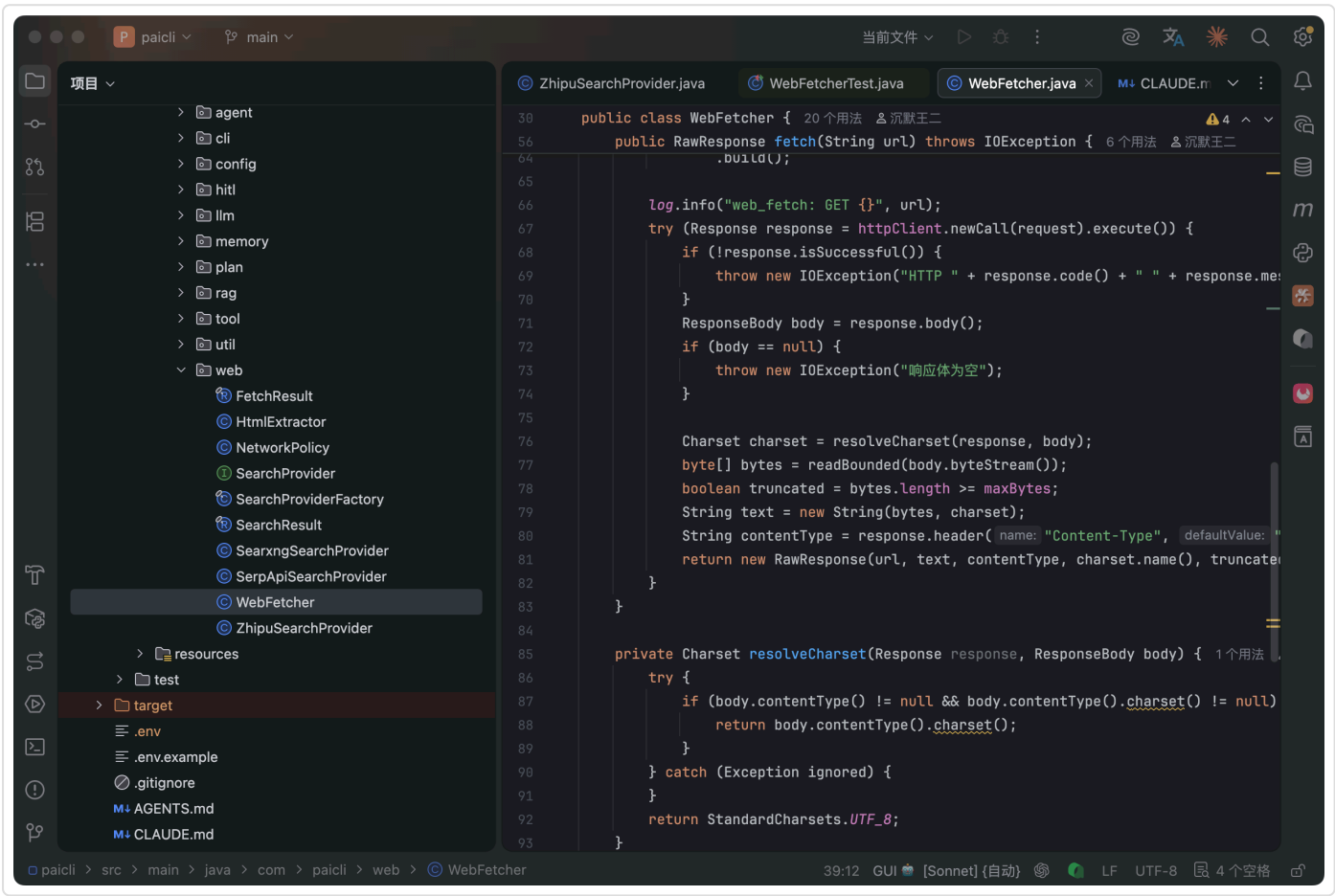
WebFetcher 负责第一步，用 OkHttp 发 GET 请求，拿回原始 HTML 字符串。几个关键参数：

响应体上限 5MB，超出会被截断。30 秒整体超时。字符集优先从 Content-Type 的 charset 参数获取，全失败用 UTF-8 兜底。

java 复制代码

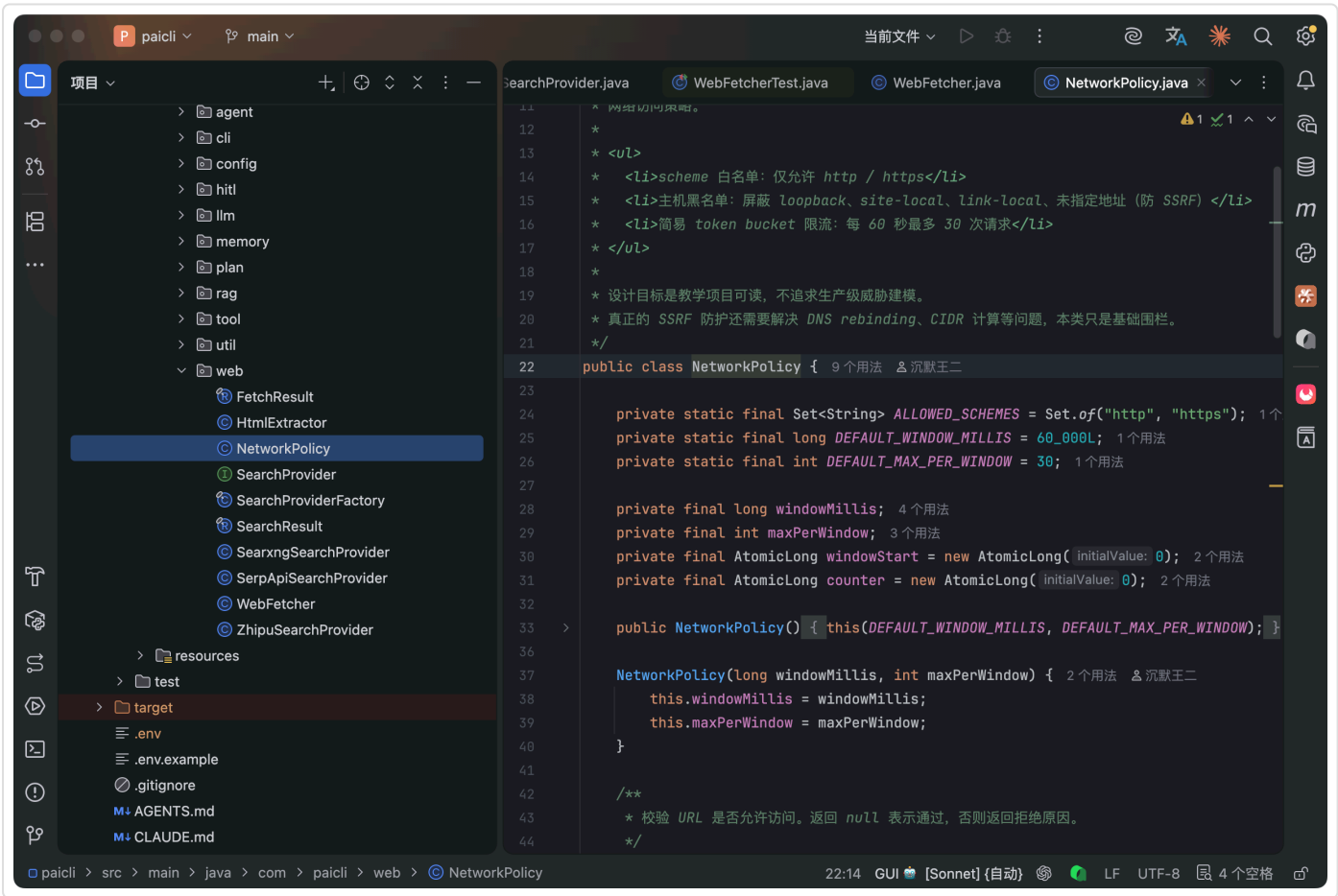
```
public RawResponse fetch(String url) throws IOException {
    Request request = new Request.Builder()
        .url(url)
        .header("User-Agent", "Mozilla/5.0 (compatible; paicli-web-fetch/1.0)")
        .get()
        .build();
    // ... 发请求，读响应，截断大体
}
```

这里有一个细节：**readBounded** 方法是流式读取的，每次读 8KB，累计到 5MB 就停。不是先把整个响应体读进内存再截断，那样碰到几百 MB 的文件会直接 OOM。



网络安全策略

在 HTTP 请求发出之前，**NetworkPolicy** 会先做两件事。

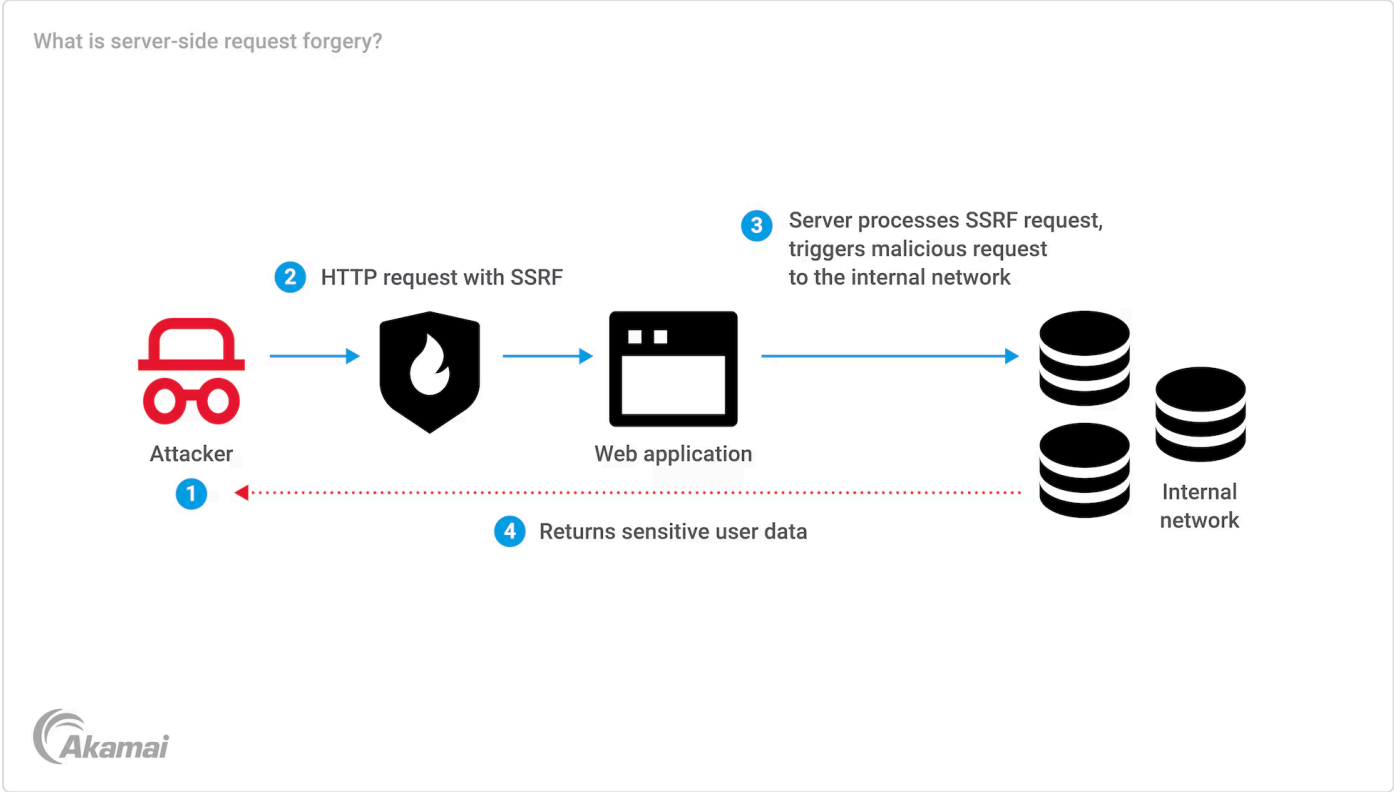


目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- 04、WebFetch 抓取器
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- 07、如何把 PaiCLI 写到简历...
[ending](#)

第一件是 URL 安全检查。只允许 http 和 https 协议，不允许 `file://` 和 `ftp://` 这些。屏蔽 localhost、127.0.0.1、内网地址（192.168.x.x、10.x.x.x），防止 SSRF 攻击。

什么是 SSRF?



假如有人让 Agent 去抓 `http://localhost:8080/admin/delete-all`，Agent 就会用自己的身份去访问你本机的服务，可能造成严重后果。

NetworkPolicy 就是防这个的。

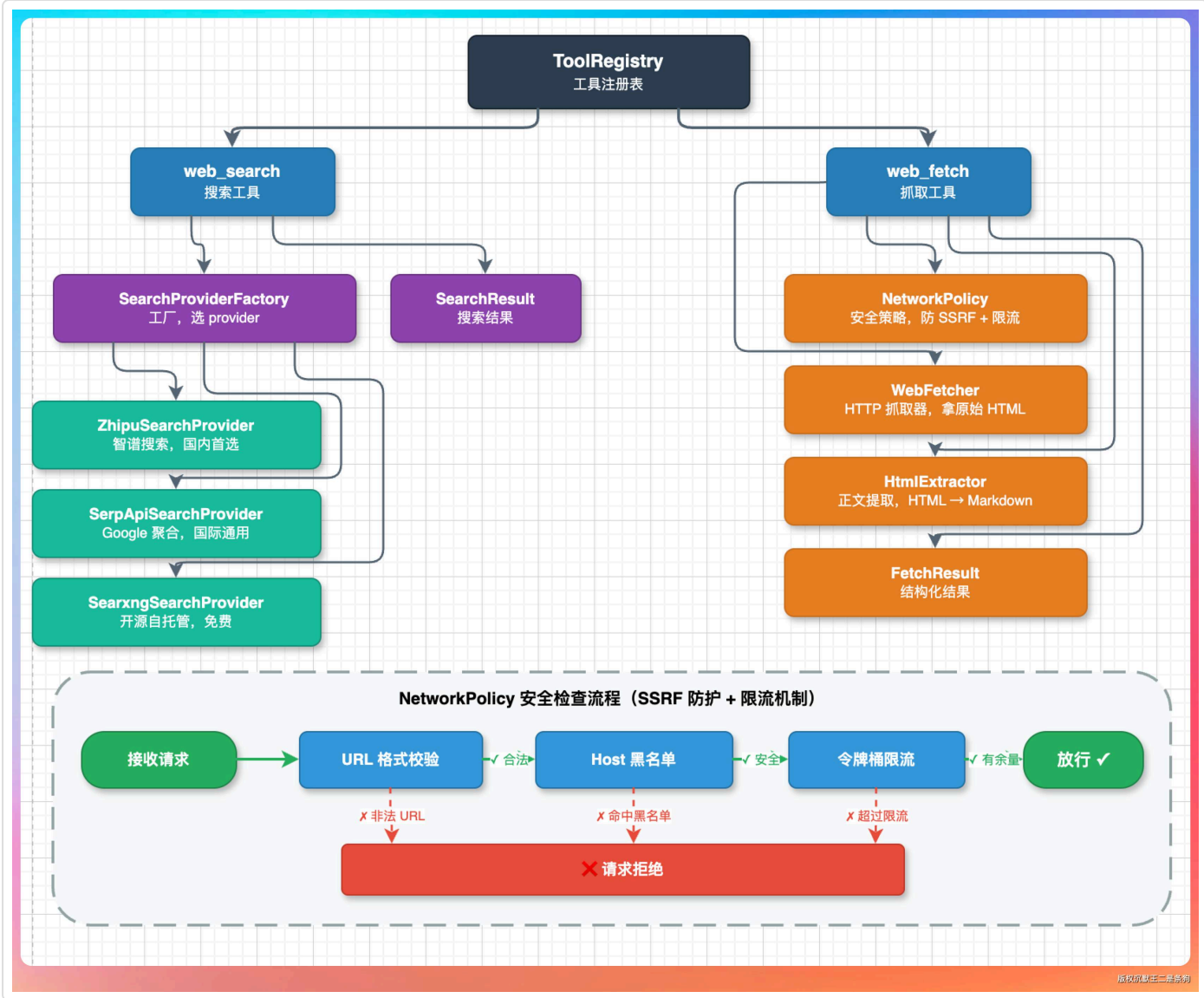
java 复制代码

```
private String checkHost(String host) {
    if (lower.equals("localhost")) return "禁止访问 localhost";
    InetAddress[] addrs = InetAddress.getAllByName(host);
    for (InetAddress addr : addrs) {
        if (addr.isLoopbackAddress()) return "禁止访问环回地址";
        if (addr.isSiteLocalAddress()) return "禁止访问站内地址";
        // ...
    }
    return null; // 通过
}
```

第二件是请求频率限制。60 秒内最多 30 次请求，超出会返回“请求过于频繁”。这个限制是为了防止 Agent 陷入重试循环，疯狂抓同一个网站被封 IP。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - 智谱搜索
 - SerpAPI
 - SearXNG
 - 工厂自动选择
- 04、WebFetch 抓取器
 - HTTP 抓取
 - 网络安全策略
 - HTML 正文提取
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - 最简配置（推荐）
 - 切换搜索引擎
 - 实际使用
- 07、如何把 PaiCLI 写到简历...ending



HTML 正文提取

HTML 里充斥着导航栏、广告、评论区、页脚这些噪声。直接把整个 HTML 丢给 LLM 既浪费 token 又影响理解。`HtmlExtractor` 的工作就是把噪声去掉，只留正文，再转成 Markdown。

提取流程分四步：

第一步，清理噪声标签。 `script`、`style`、`nav`、`aside`、`footer`、`header`、`form`、`iframe` 这些标签直接删掉。`class` 或 `id` 里包含 `ads`、`banner`、`sidebar`、`comment` 等关键词的元素也一并清掉。

第二步，找主语义容器。 优先找 `<article>`、`<main>`、`[role=main]` 这些语义化标签。大部分博客和文档站都有这些标签，找到就能直接定位到正文区域。

第三步，打分兜底。 如果页面没有语义化标签（很多老网站就是一堆 `div` 嵌套），就给所有 `block` 元素打分。打分公式很简洁： $\text{文本长度} \times (1 - \text{链接密度惩罚})$ 。文本越多、链接占比越低的元素，越可能是正文。

第四步，转 Markdown。 把选中的正文容器递归遍历，`h1-h6` 转标题、`p` 转段落、`strong` 转粗体、`a` 转链接、`pre/code` 转代码块、`table` 转 Markdown 表格。

java 复制代码

```
private double score(Element el) {
    String text = el.text();
    int textLen = text.length();
    if (textLen < 80) return 0;
    int linkLen = 0;
    for (Element a : el.select("a")) {
        linkLen += a.text().length();
    }
    double linkRatio = (double) linkLen / textLen;
    double penalty = Math.min(linkRatio * 2.0, 1.0);
    return textLen * (1.0 - penalty);
}
```

这个打分公式的思路是：导航栏和侧边栏的特征是“链接密度高”，正文的特征是“文字多、链接少”。用链接密度做惩罚，就能比较准确地区分二者。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - 智谱搜索
 - SerpAPI
 - SearXNG
 - 工厂自动选择
- 04、WebFetch 抓取器
 - HTTP 抓取
 - 网络安全策略
 - HTML 正文提取
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - 最简配置（推荐）
 - 切换搜索引擎
 - 实际使用
- 07、如何把 PaiCLI 写到简历...ending

原始HTML输入

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>技术博客 - AI发展</title>
5   <meta charset="UTF-8">
6   <script>
7     // 广告追踪代码
8     analytics.track('page_view');
9   </script>
10  <style>
11    body { font-family: Arial; }
12    .ad-banner { display: block; }
13  </style>
14 </head>
15 <body>
16   <nav>
17     <a href="/">首页</a>
18     <a href="/about">关于</a>
19   </nav>
20   <div class="ad-banner">
21     
22     <p>点击购买优惠商品!</p>
23   </div>
24   <article>
25     <h1>人工智能的发展历程</h1>
26     <p>发布时间: 2024-01-15</p>
27     <p>作者: 张三</p>
28     <div class="content">
29       <p>人工智能(AI)是计算机科学的一个分支...</p>
30       <p>近年来,深度学习技术的突破使得AI在图像识别、自然语言
31       
32       <p>未来,AI将在更多领域发挥作用...</p>
33     </div>
34   </article>
35   <footer>
36     <p>© 2024 技术博客</p>
37     <div class="social-links">
38       <a href="#">微博</a>
39       <a href="#">微信</a>
40     </div>
41   </footer>
42 </body>
43 </html>
```

HtmlExtractor 处理流程

1 解析HTML

2 噪声清理

- 移除script标签
- 移除style标签
- 移除广告元素
- 移除导航栏
- 移除页脚

3 正文提取

- 识别article标签
- 提取标题
- 提取段落
- 提取图片

4 Markdown转换

- h1 → #
- p → 段落
- img → ![]()

5 输出结果

提取后的Markdown

```
1 # 人工智能的发展历程
2
3 发布时间: 2024-01-15
4 作者: 张三
5
6 人工智能(AI)是计算机科学的一个分支,致力于创造能够
7 模拟人类智能的系统。从早期的专家系统到现代的深度学习,
8 AI技术经历了长足的发展。
9
10 近年来,深度学习技术的突破使得AI在图像识别、自然语言
11 处理等领域取得了显著进展。神经网络模型的规模不断增大,
12 训练数据量也呈指数级增长。
13
14 ![AI示意图](ai.jpg)
15
16 未来,AI将在更多领域发挥作用,包括医疗诊断、自动驾驶、
17 教育辅助等。同时,我们也需要关注AI伦理和安全问题。
```

转换效果统计对比

	原始HTML	VS	提取后Markdown
字符数	1,247		312
行数	45		12
噪声比例	75%		0%
提取率	-		25%
信噪比	1:3		1:0

核心特性

- 智能噪声过滤
- 正文精准提取
- 结构化Markdown输出
- 保留关键元信息
- 支持多种HTML格式

HtmlExtractor 有一个已知的边界：JS 渲染的 SPA 页面（比如 React/Vue 单页应用）抓回来可能是空白的，因为正文要靠 JavaScript 执行才能生成。

另外一些网站有反爬机制，比如 Cloudflare 的人机验证页面，抓回来也是一堆验证脚本而不是真正的内容。

这个问题会在接入 Chrome DevTools MCP 后解决，到时候会用真正的浏览器去渲染页面，JS 该跑跑、验证码该过过。

目前遇到空正文会返回一条提示：**"未提取到正文。可能是 JS 渲染或防爬墙；本期范围内不再重试。"**，让 Agent 知道这是已知边界，不要反复重试浪费 token。

05、工具注册和 Agent 集成

工具实现好了，还需要注册到 **ToolRegistry** 里，Agent 才能发现和使用它们。

ToolRegistry 是 PaiCLI 的工具中心，所有工具（文件操作、Shell 命令、RAG 检索、联网工具）都在这里注册。注册一个工具需要四样东西：名称、描述、参数定义、执行函数。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象

智谱搜索

SerpAPI

SearXNG

工厂自动选择
- 04、WebFetch 抓取器

HTTP 抓取

网络安全策略

HTML 正文提取
- 05、工具注册和 Agent 集成
- 06、配置和使用

最简配置（推荐）

切换搜索引擎

实际使用
- 07、如何把 PaiCLI 写到简历...ending

```
private void registerWebTools() {
    tools.put("web_search", new Tool(
        "web_search",
        "搜索互联网，获取实时信息...",
        createParameters(
            new Param("query", "string", "搜索关键词", true),
            new Param("top_k", "integer", "返回结果数量（默认5）", false)
        ),
        args -> webSearch(args.get("query"), parseInt(args.get("top_k"), 5))
    ));

    tools.put("web_fetch", new Tool(
        "web_fetch",
        "抓取指定 URL，提取正文转 Markdown...",
        createParameters(
            new Param("url", "string", "完整 URL", true),
            new Param("max_chars", "integer", "最大字符数（默认 8000）", false)
        ),
        args -> webFetch(args.get("url"), parseInt(args.get("max_chars"), 8000))
    ));
}
```

这里的描述文本很重要，是给 LLM 看的。

LLM 根据工具描述来决定什么时候用什么工具。所以 `web_search` 的描述里写了“获取实时信息（最新版本、官方文档、技术资讯等）”，给 LLM 明确的使用场景提示。

`web_fetch` 的描述里写了“适用静态/SSR 页面”和“JS 渲染或防爬站会返回空正文”，让 LLM 知道边界在哪。

目录

- 01、Agent 的联网搜索效果
- 02、Agent 联网搜索的整体架...
- 03、搜索引擎 Provider 抽象
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- 04、WebFetch 抓取器
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- 05、工具注册和 Agent 集成
- 06、配置和使用
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- 07、如何把 PaiCLI 写到简历...ending

```
/**
 * 工具注册表 - 管理所有可用工具
 */
public class ToolRegistry {
    private static final ObjectMapper mapper = new ObjectMapper();
    private static final int DEFAULT_COMMAND_TIMEOUT_SECONDS = 60;
    private static final int DEFAULT_TOOL_BATCH_TIMEOUT_SECONDS = 90;
    private static final int MAX_PARALLEL_TOOLS = 4;
    private static final int MAX_COMMAND_OUTPUT_CHARS = 8_000;
    private final Map<String, Tool> tools = new HashMap<>();
    private final long commandTimeoutSeconds;
    private final long toolBatchTimeoutSeconds;
    private static final int DEFAULT_FETCH_MAX_CHARS = 8_000;
    private String projectPath = System.getProperty("user.dir");
    private SearchProvider searchProvider;
    private WebFetcher webFetcher;
    private HtmlExtractor htmlExtractor;
    private NetworkPolicy networkPolicy;

    public ToolRegistry() {
        this(DEFAULT_COMMAND_TIMEOUT_SECONDS, DEFAULT_TOOL_BATCH_TIMEOUT_SECONDS);
    }

    ToolRegistry(long commandTimeoutSeconds) {
        this(commandTimeoutSeconds, Math.max(commandTimeoutSeconds + 5, DEFAULT_TOOL_BATCH_TIMEOUT_SECONDS));
    }

    ToolRegistry(long commandTimeoutSeconds, long toolBatchTimeoutSeconds) {
        this.commandTimeoutSeconds = commandTimeoutSeconds;
        this.toolBatchTimeoutSeconds = toolBatchTimeoutSeconds;
        // 注册内置工具
        registerFileTools();
        registerShellTools();
    }
}
```

SearchProvider 和 WebFetcher 这些组件都是懒加载的——第一次用到的时候才创建实例。因为不是每次对话都需要联网，没必要一起动就初始化网络组件。


```
private synchronized SearchProvider searchProvider() {
    if (searchProvider == null) {
        searchProvider = SearchProviderFactory.create();
    }
    return searchProvider;
}
```

java 复制代码

加了 `synchronized` 是因为 PaiCLI 支持并行工具调用，多个工具可能同时执行，需要保证只初始化一次。

不加这个关键字的话，两个线程同时进来可能会创建两个 SearchProvider 实例，虽然不会报错但浪费资源，而且状态可能不一致。这是 Java 并发编程里面非常经典的双重检查锁定场景。

06、配置和使用

说完了实现原理，讲讲怎么配置和使用。

最简配置（推荐）

如果你已经有 `GLM_API_KEY`（用来跑 GLM-5.1 模型的），恭喜你，不需要任何额外配置。

PaiCLI 会自动检测到 `GLM_API_KEY`，用智谱搜索作为默认搜索引擎。

在项目根目录的 `.env` 文件里确认一下有这行就行：

```
GLM_API_KEY=你的智谱API密钥
```

ini 复制代码

```
# ===== Web 搜索配置 =====
# 选择搜索 provider (zhipu | serpapi | searxng)，不配置时按 Key/URL 自动判断：
# 1. 有 GLM_API_KEY → zhipu（默认推荐：与 GLM 推理共用 Key，零额外配置，国内首选）
# 2. 有 SERPAPI_KEY → serpapi（国际通用，付费即开即用）
# 3. 有 SEARXNG_URL → searxng（开源自托管，免费，需本地跑 docker 实例）
# SEARCH_PROVIDER=zhipu

# 智谱 Web Search 引擎（SEARCH_PROVIDER=zhipu 时启用）
# 可选：search_std（0.01 元/次，默认）、search_pro（0.03 元/次）、
#       search_pro_sogou（0.05 元/次，搜狗）、search_pro_quark（0.05 元/次，夸克）
# 中文搜索建议用 search_pro 或搜狗 / 夸克版本，效果优于通用 std
# ZHIPU_SEARCH_ENGINE=search_std

# SerpAPI Key（SEARCH_PROVIDER=serpapi 时启用）
# 免费 100 次/月：https://serpapi.com/manage-api-key
# SERPAPI_KEY=your_serpapi_key_here

# SearXNG 实例地址（SEARCH_PROVIDER=searxng 时启用）
# 本地 docker：docker run --rm -p 8888:8888 searxng/searxng
# 公共实例（不稳定，仅试玩）：见 https://searx.space
# SEARXNG_URL=http://localhost:8888

# ===== Web 抓取配置 =====
# web_fetch 工具：抓取 URL 并提取正文 Markdown
# 当前实现走「直接 HTTP + Jsoup + 简易 readability」，对静态/SSR 页面有效
# 对 SPA / 防爬墙站点会返回空正文（已知边界，待第 13/14 期 CDP 路线接入）
# 默认安全策略：屏蔽 file:// / 内网 / loopback；30 秒超时；5MB 响应上限；每分钟 30 次限流
```

版权属默庄二是条狗

切换搜索引擎

如果你想用 SerpAPI（国际搜索能力更强），在 `.env` 里加两行：

```
SEARCH_PROVIDER=serpapi
SERPAPI_KEY=你的SerpAPI密钥
```

ini 复制代码

目录

[01、Agent 的联网搜索效果](#)

[02、Agent 联网搜索的整体架...](#)

[03、搜索引擎 Provider 抽象](#)

[智谱搜索](#)

[SerpAPI](#)

[SearXNG](#)

[工厂自动选择](#)

[04、WebFetch 抓取器](#)

[HTTP 抓取](#)

[网络安全策略](#)

[HTML 正文提取](#)

[05、工具注册和 Agent 集成](#)

[06、配置和使用](#)

[最简配置（推荐）](#)

[切换搜索引擎](#)

[实际使用](#)

[07、如何把 PaiCLI 写到简历...](#)

[ending](#)

如果你想用免费的 SearXNG，先启动 Docker 实例，再配置：

```
docker run --rm -p 8888:8888 searxng/searxng
```

bash 复制代码

```
SEARCH_PROVIDER=searxng
SEARXNG_URL=http://localhost:8888
```

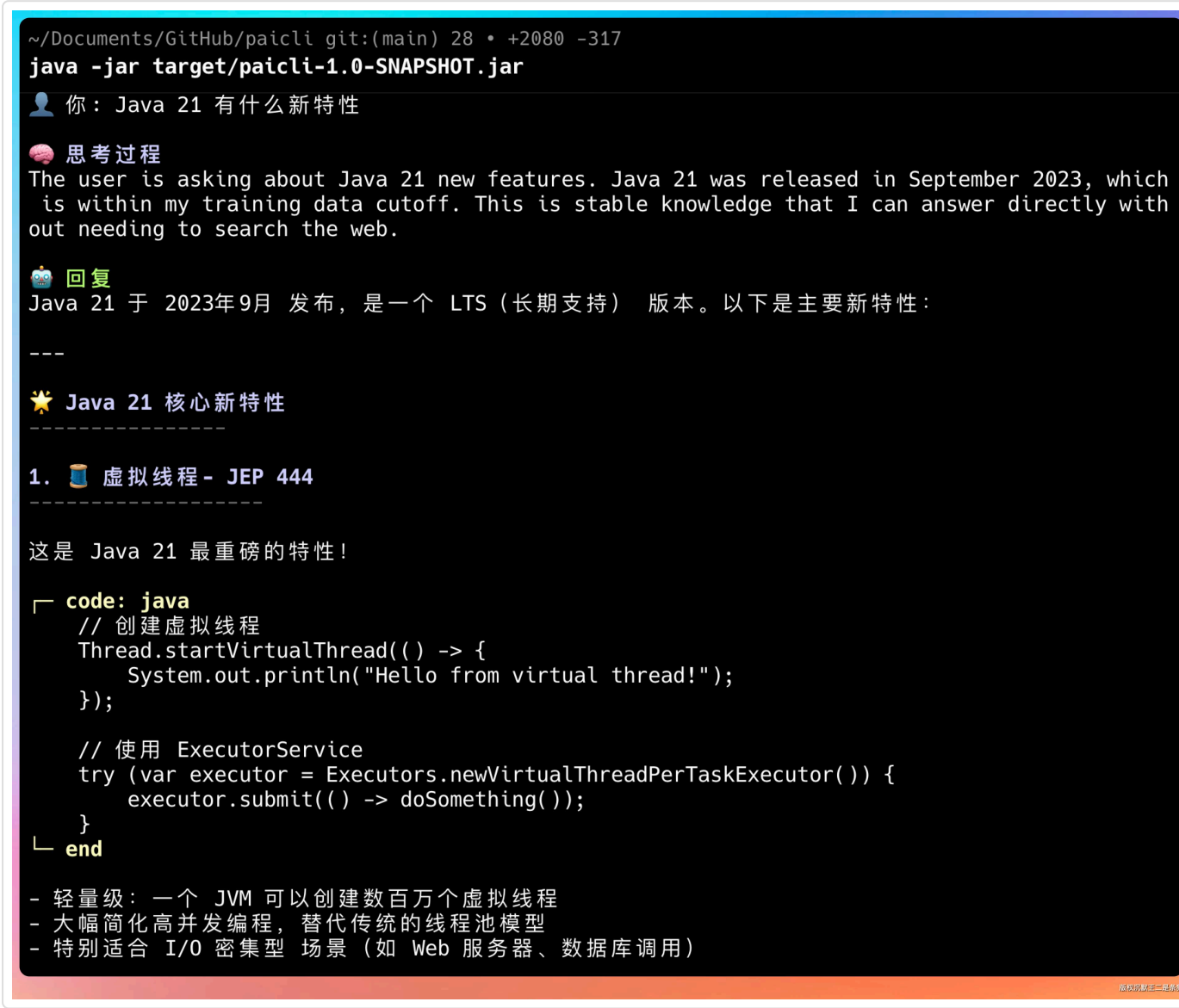
ini 复制代码

实际使用

启动 PaiCLI 后，Agent 会自动识别哪些问题需要联网。你不需要手动指定“用搜索工具”，Agent 自己会判断。

几个典型的使用场景：

问“Java 21 有什么新特性”，Agent 会调用 `web_search` 去搜索最新信息。



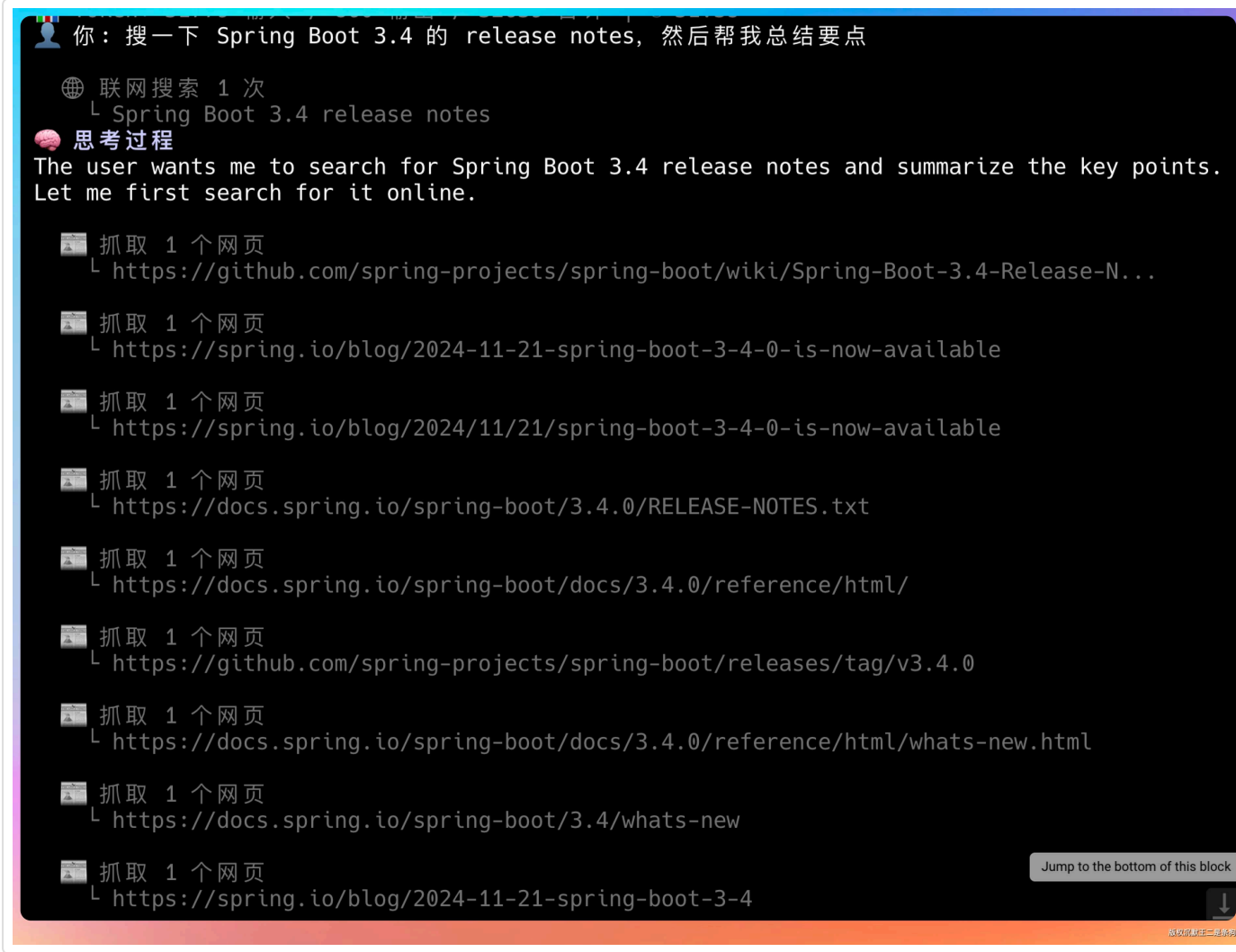
问“帮我看看 paicoding.com 首页有什么内容”，Agent 会调用 `web_fetch` 去抓取页面。

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- [04、WebFetch 抓取器](#)
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- [07、如何把 PaiCLI 写到简历...ending](#)



问“搜一下 Spring Boot 3.4 的 release notes，然后帮我总结要点”，Agent 会先搜再抓，组合使用两个工具。



这里有个小技巧：如果你觉得搜索结果不够详细，可以追问一句“帮我打开第一条链接看看详细内容”，Agent 就会自动用 `web_fetch` 去抓取搜索结果里的 URL。搜索 + 抓取的组合拳打法，基本上能覆盖百分之 80 的联网需求了。

07、如何把 PaiCLI 写到简历上？

学完这一期，大家可以在简历上这样写：

- **项目名称**：PaiCLI - Java Agent CLI
- **项目简介**：从零构建的生产级 Java Agent 命令行工具，支持联网搜索、网页抓取、RAG 检索、多 Agent 协作等能力

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- [04、WebFetch 抓取器](#)
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- [07、如何把 PaiCLI 写到简历...ending](#)

- **技术栈：**Java 21、OkHttp、Jsoup、GLM-5.1/DeepSeek V4、策略模式、工厂模式
- **核心职责：**
 - 基于策略模式设计了 SearchProvider 搜索引擎抽象层，支持智谱/SerpAPI/SearXNG，可在运行时自动选择和热切换
 - 实现了基于 Jsoup 的 Readability 正文提取算法，通过语义标签优先+链接密度评分的两阶段策略准确提取网页正文
 - 设计 NetworkPolicy 网络安全策略，包括 SSRF 防护（scheme 白名单+host 黑名单+DNS 解析校验）和令牌桶限流
 - 基于工厂模式实现了 SearchProviderFactory，支持从环境变量、系统属性、.env 文件三级回退读取配置，实现零额外配置的开箱即用体验
 - 将 web_search 和 web_fetch 作为 Function Calling 工具注册到 Agent 的工具链中，实现了 LLM 自主判断联网时机的智能工具选择

项目源码地址：<https://github.com/itwanger/paicli>，第 9 期的代码已经全部提交。

欢迎大家 star、fork、提 issue，一起把这个项目做得更好。

ending

从第 1 期的 400 行 ReAct 循环，到现在第 9 期加上联网能力，PaiCLI 已经不再是一个“只会操作本地文件”的 Agent 了。

搜索让它知道世界上正在发生什么，

抓取让它能真正读懂一个网页在说什么，

安全策略让它不会乱来。

如果你也想从零开始理解 Agent 是怎么一步步搭建起来的，



跟着 PaiCLI 的路线图走就对了。

每一期都有完整代码、有真实 case、有可以写进简历的亮点。

【最好的学习方式不是看别人怎么用，而是自己从零造一个。】

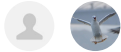
我们下期见。

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- [04、WebFetch 抓取器](#)
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- [07、如何把 PaiCLI 写到简历...](#)
- [ending](#)



2人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

- [01、Agent 的联网搜索效果](#)
- [02、Agent 联网搜索的整体架...](#)
- [03、搜索引擎 Provider 抽象](#)
 - [智谱搜索](#)
 - [SerpAPI](#)
 - [SearXNG](#)
 - [工厂自动选择](#)
- [04、WebFetch 抓取器](#)
 - [HTTP 抓取](#)
 - [网络安全策略](#)
 - [HTML 正文提取](#)
- [05、工具注册和 Agent 集成](#)
- [06、配置和使用](#)
 - [最简配置（推荐）](#)
 - [切换搜索引擎](#)
 - [实际使用](#)
- [07、如何把 PaiCLI 写到简历...ending](#)